

Reflekt - AI Journal App

Product Requirements Document v3.0

App Overview

Field	Detail
App Name	Reflekt
Platform	React Native (Expo)
Backend	Node.js + Express
Database	PostgreSQL (raw, no ORM)
Build Time	3 Weeks
Total Cost	\$0/month
Purpose	AI-powered daily journal with voice-to-text, mood tracking, and weekly AI summaries

Tech Stack

Layer	Technology	Cost
Mobile App	React Native (Expo)	Free
Backend	Node.js + Express	Free
Database	PostgreSQL (self-hosted)	Free
Auth	JWT + bcrypt (custom)	Free
Voice to Text	Whisper.cpp (self-hosted)	Free
AI Summary	Ollama + Llama 3.1 (self-hosted)	Free
Audio Recording	expo-av	Free
File Storage	Local disk on server	Free
Push Notifications	Expo Notifications	Free
Hosting	Oracle Cloud Free Tier	Free

Free AI Infrastructure Setup

Whisper.cpp (Voice to Text)

- Open source speech recognition by OpenAI, runs entirely on your server
- No API cost, no rate limits, no internet dependency after install
- Install command: `git clone https://github.com/ggerganov/whisper.cpp`
`&& cd whisper.cpp && make`
- Recommended model: `ggml-base.en.bin` — good balance of speed and accuracy
- Called from Node.js via `child_process.exec()`

Ollama (AI Summaries)

- Runs large language models locally on your server
- Install command: `curl -fsSL https://ollama.com/install.sh | sh`
- Primary model: `ollama pull llama3.1` — 8B parameters, fast, high quality
- Fallback model: `ollama pull mistral` — smaller and faster on low RAM
- Called via HTTP POST to `http://localhost:11434/api/generate`
- No API key, no token costs, no rate limits

Oracle Cloud Free Tier (Hosting)

- 2 AMD vCPUs, 24GB RAM, 200GB storage — free forever
- Sufficient to run PostgreSQL, Node.js, Ollama, and Whisper.cpp simultaneously
- Sign up at cloud.oracle.com

Project Folder Structure

Backend Structure

backend/

src/

index.js -- Express app entry, middleware setup, route registration

config/

db.js -- PostgreSQL connection pool (pg library)

jwt.js -- JWT sign and verify helpers

routes/

auth.js -- POST /register, POST /login, GET /me

entries.js -- CRUD for journal entries

transcribe.js -- POST /transcribe (audio to text)

summaries.js -- GET and POST for weekly AI summaries

streaks.js -- GET streak data

settings.js -- GET and PATCH user settings

controllers/

 authController.js -- Register, login, me logic

 entriesController.js -- Create, read, update, delete entries

 transcribeController.js -- Audio upload and Whisper call

 summariesController.js -- Fetch and generate weekly summaries

 streaksController.js -- Return streak data for user

 settingsController.js -- Read and update user preferences

middleware/

 authMiddleware.js -- Verify JWT on every protected route

 uploadMiddleware.js -- Multer config for audio file uploads

services/

 whisperService.js -- Subprocess call to whisper.cpp binary

 ollamaService.js -- HTTP call to Ollama generate endpoint

 streakService.js -- Streak calculation and update logic

migrations/

 001_create_users.sql

 002_create_journal_entries.sql

 003_create_weekly_summaries.sql

004_create_streaks.sql

005_create_user_settings.sql

uploads/ -- Audio files stored here temporarily

package.json

.env

Frontend Structure

mobile/

src/

screens/

OnboardingScreen.tsx

RegisterScreen.tsx

LoginScreen.tsx

HomeScreen.tsx

NewEntryScreen.tsx

EditEntryScreen.tsx

EntryDetailScreen.tsx

JournalListScreen.tsx

InsightsScreen.tsx

SettingsScreen.tsx

components/

EntryCard.tsx -- Single journal entry list item

MoodSelector.tsx -- 5-emoji mood picker

MoodChip.tsx -- Filter chip for mood filtering

StreakBadge.tsx -- Streak count display
WeeklySummaryCard.tsx -- AI summary display card
MoodChart.tsx -- Line chart for 7-day mood trend
MoodPieChart.tsx -- Pie chart for mood distribution
WaveformAnimation.tsx -- Animated recording indicator

services/

ApiService.ts -- Axios base instance with JWT header injection
AuthService.ts -- Register, login, me API calls
JournalService.ts -- Entry CRUD API calls
AudioService.ts -- expo-av record and playback
SummaryService.ts -- Summary fetch and generate API calls
StreakService.ts -- Streak fetch API call
NotificationService.ts -- Expo local notification scheduler

navigation/

AppNavigator.tsx -- Root stack navigator
BottomTabNavigator.tsx -- Home, Journal, Insights, Settings tabs

store/

authStore.ts -- Zustand store for user session
journalStore.ts -- Zustand store for entries cache

utils/

encryption.ts -- AES-256 encrypt and decrypt helpers

App.tsx

Database Schema

-- =====

-- TABLE: users

-- =====

```
CREATE TABLE users (  
  id          SERIAL PRIMARY KEY,  
  full_name   VARCHAR(100) NOT NULL,  
  email       VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  avatar_url  TEXT,  
  created_at  TIMESTAMP DEFAULT NOW()  
);
```

-- =====

-- TABLE: journal_entries

-- =====

```
CREATE TABLE journal_entries (  
  id          SERIAL PRIMARY KEY,  
  user_id     INTEGER REFERENCES users(id) ON DELETE CASCADE,  
  entry_date  DATE NOT NULL,  
  mood        INTEGER CHECK (mood BETWEEN 1 AND 5),  
  content     TEXT NOT NULL,  
  audio_path  TEXT,  
  tags        TEXT[],
```

```
    created_at  TIMESTAMP DEFAULT NOW(),
    updated_at  TIMESTAMP DEFAULT NOW()
);
```

```
-- =====
```

```
-- TABLE: weekly_summaries
```

```
-- =====
```

```
CREATE TABLE weekly_summaries (
    id          SERIAL PRIMARY KEY,
    user_id     INTEGER REFERENCES users(id) ON DELETE CASCADE,
    week_start  DATE NOT NULL,
    summary     TEXT NOT NULL,
    top_themes  TEXT[],
    avg_mood    DECIMAL(3,2),
    created_at  TIMESTAMP DEFAULT NOW(),
    UNIQUE(user_id, week_start)
);
```

```
-- =====
```

```
-- TABLE: streaks
```

```
-- =====
```

```
CREATE TABLE streaks (
    id          SERIAL PRIMARY KEY,
    user_id     INTEGER REFERENCES users(id) ON DELETE CASCADE UNIQUE,
```

```

current_streak    INTEGER DEFAULT 0,
longest_streak    INTEGER DEFAULT 0,
last_entry_date   DATE,
updated_at        TIMESTAMP DEFAULT NOW()
);

-- =====

-- TABLE: user_settings

-- =====

CREATE TABLE user_settings (
    user_id         INTEGER PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    reminder_enabled BOOLEAN DEFAULT FALSE,
    reminder_time    TIME,
    updated_at       TIMESTAMP DEFAULT NOW()
);

-- =====

-- INDEXES

-- =====

CREATE INDEX idx_entries_user_date ON journal_entries(user_id, entry_date DESC);

CREATE INDEX idx_summaries_user_week ON weekly_summaries(user_id, week_start
DESC);

```

Environment Variables

Backend (.env)

PORT=3000

DATABASE_URL=postgresql://user:password@localhost:5432/reflekt

JWT_SECRET=your_long_random_secret_here

JWT_EXPIRES_IN=30d

UPLOADS_DIR=./uploads

WHISPER_MODEL_PATH=./models/ggml-base.en.bin

WHISPER_BINARY=./whisper.cpp/main

OLLAMA_URL=http://localhost:11434

OLLAMA_MODEL=llama3.1

Mobile (.env)

API_BASE_URL=http://your-server-ip:3000/api

MODULE 1: AUTH MODULE

Overview

Handles user registration, login, and session management using JWT tokens and bcrypt password hashing. No third-party auth service. Everything runs on your own server.

Backend Files

- `src/routes/auth.js`

- `src/controllers/authController.js`
- `src/config/jwt.js`
- `src/middleware/authMiddleware.js`

Frontend Files

- `screens/RegisterScreen.tsx`
 - `screens/LoginScreen.tsx`
 - `services/AuthService.ts`
 - `store/authStore.ts`
 - `navigation/AppNavigator.tsx` (AuthGate logic)
-

Screen: Register Screen

Route: `/register`

User Story

As a new user, I want to create an account with my name, email, and password so I can start journaling privately and securely.

UI Elements

- App logo centered at top
- Full name text input
- Email text input with keyboard type email
- Password text input with secure entry toggle
- Register button (primary)
- "Already have an account? Login" link at bottom

Frontend Logic

1. User fills in all three fields and taps Register
2. Validate: all fields required, email format valid, password min 8 characters
3. Show inline error messages per field on validation failure
4. On valid form, show loading spinner on button
5. Call `AuthService.register({ full_name, email, password })`
6. On success, store returned JWT in `expo-secure-store`
7. Set user in `authStore`

8. Navigate to Home Screen
9. On API error (e.g. email already exists), show error message below email field

API Call

POST /api/auth/register

Body: { full_name, email, password }

Success Response: { token, user: { id, full_name, email } }

Error Response: { error: "Email already registered" }

Screen: Login Screen

Route: /login

User Story

As a returning user, I want to log in with my email and password so I can access my journal entries.

UI Elements

- App logo
- Email and password inputs
- Login button
- Forgot password link (shows alert: "Reset via email - coming soon")
- "Don't have an account? Register" link

Frontend Logic

1. User enters email and password and taps Login
2. Validate: both fields required, email format valid
3. Call `AuthService.login({ email, password })`
4. On success, store JWT in `expo-secure-store`, set authStore, navigate to Home
5. On 401 error, show "Incorrect email or password" message
6. On network error, show "Cannot connect to server" message

API Call

POST /api/auth/login

Body: { email, password }

Success Response: { token, user: { id, full_name, email } }

Error Response: { error: "Invalid credentials" }

API Routes: Auth Module

POST /api/auth/register

User Story: As a new user, I want to register so the system creates my account and gives me a token to use the app without logging in every time.

Method: POST

Route: /api/auth/register

Auth: None required

Body: { full_name, email, password }

Response: { token, user: { id, full_name, email } }

Step-by-step Logic:

1. Receive `full_name`, `email`, `password` from request body
2. Validate all three fields are present, return 400 if any missing
3. Query: `SELECT id FROM users WHERE email = $1` — return 409 if email exists
4. Hash password: `bcrypt.hash(password, 10)`
5. Insert: `INSERT INTO users (full_name, email, password_hash) VALUES ($1, $2, $3) RETURNING id`
6. Insert default streak row: `INSERT INTO streaks (user_id, current_streak) VALUES ($1, 0)`
7. Insert default settings row: `INSERT INTO user_settings (user_id) VALUES ($1)`

8. Sign JWT: `jwt.sign({ userId: user.id }, JWT_SECRET, { expiresIn: '30d' })`
9. Return `{ token, user: { id, full_name, email } }`

Error Responses:

- 400: Missing required fields
 - 409: Email already registered
 - 500: Database or server error
-

POST /api/auth/login

User Story: As a registered user, I want to log in so I get a fresh token and can access my private journal data.

Method: POST

Route: /api/auth/login

Auth: None required

Body: { email, password }

Response: { token, user: { id, full_name, email } }

Step-by-step Logic:

1. Receive `email` and `password` from request body
2. Validate both fields present, return 400 if missing
3. Query: `SELECT * FROM users WHERE email = $1`
4. Return 401 if no user found — use message "Invalid credentials" (do not reveal which field is wrong)
5. Compare: `bcrypt.compare(password, user.password_hash)`
6. Return 401 if comparison fails
7. Sign fresh JWT with `userId`
8. Return `{ token, user: { id, full_name, email } }`

Error Responses:

- 400: Missing fields
- 401: Invalid credentials

- 500: Server error
-

GET /api/auth/me

User Story: As a logged-in user, I want the app to load my profile automatically on launch so I do not have to log in every time I open the app.

Method: GET

Route: /api/auth/me

Auth: Bearer token required

Response: { id, full_name, email, avatar_url, created_at }

Step-by-step Logic:

1. `authMiddleware` extracts and verifies JWT from `Authorization: Bearer <token>` header
2. Middleware attaches `req.userId` from token payload
3. Query: `SELECT id, full_name, email, avatar_url, created_at FROM users WHERE id = $1`
4. Return 404 if user not found (deleted account edge case)
5. Return user object

Error Responses:

- 401: Missing or invalid token
 - 404: User not found
 - 500: Server error
-

MODULE 2: JOURNAL CRUD MODULE

Overview

Core module. Handles creating, reading, updating, and deleting journal entries. All content is encrypted on device with AES-256 before being sent to the server. The server stores only the encrypted blob and never sees plaintext.

Backend Files

- `src/routes/entries.js`
- `src/controllers/entriesController.js`

Frontend Files

- `screens/NewEntryScreen.tsx`
 - `screens/EditEntryScreen.tsx`
 - `screens/EntryDetailScreen.tsx`
 - `screens/JournalListScreen.tsx`
 - `components/EntryCard.tsx`
 - `services/JournalService.ts`
 - `utils/encryption.ts`
 - `store/journalStore.ts`
-

Screen: New Entry Screen

Route: `/entry/new`

User Story

As a user, I want to write or speak my journal entry for today, select my mood, and add tags so I can track how I feel over time.

UI Elements

- Back button top left
- Today's date displayed top center
- Mood selector: 5 emoji buttons (1=awful, 2=bad, 3=okay, 4=good, 5=amazing)
- Large multi-line text input taking most of screen
- Microphone button bottom center (triggers voice overlay)
- Tag chips row: Work, Personal, Health, Relationships, Other
- Save button top right

Frontend Logic

1. User selects mood by tapping one of the 5 emojis (required before save)
2. User types text directly or taps mic button to use voice
3. User taps one or more tag chips to categorize entry
4. On Save tap:
 - Validate mood is selected and content is not empty
 - Encrypt content using `encryption.ts` AES-256 with key from SecureStore
 - Call `JournalService.createEntry({ entry_date, mood, content: encryptedContent, tags, audio_path })`
 - On success, update `journalStore`, navigate back to Home
5. Show loading state on Save button during API call

API Call

POST /api/entries

Body: { entry_date, mood, content (encrypted), tags[], audio_path }

Success Response: { id, entry_date, mood, content, tags, created_at }

Screen: Journal List Screen

Route: `/journal`

User Story

As a user, I want to browse all my past journal entries with filters for mood and tags so I can find specific entries quickly.

UI Elements

- Search bar at top
- Mood filter chips row (show only entries matching selected mood)
- Tag filter chips row
- Scrollable chronological list of EntryCard components
- Each card shows: date, mood emoji, first 2 lines of content (decrypted), tags
- Pull to refresh gesture

Frontend Logic

1. On mount, call `JournalService.getEntries()`
2. Decrypt content of each entry using `encryption.ts` before rendering
3. When mood chip tapped, re-fetch with `?mood=X` query param
4. When tag chip tapped, re-fetch with `?tag=X` query param
5. Search bar filters decrypted content locally without re-fetching
6. Pull to refresh triggers fresh fetch and re-decrypt
7. Tap on any card navigates to Entry Detail Screen with entry id

API Call

GET `/api/entries?mood=3&tag=work&search=keyword`

Response: [{ id, entry_date, mood, content (encrypted), audio_path, tags }]

Screen: Entry Detail Screen

Route: `/entry/:id`

User Story

As a user, I want to view the full content of a specific past entry so I can reflect on what I wrote and how I felt that day.

UI Elements

- Back button
- Date and mood emoji at top
- Full decrypted entry content in scrollable view
- Tag chips below content
- Play audio button (visible only if `audio_path` exists)
- Edit button top right
- Delete button bottom (red, with confirmation alert)

Frontend Logic

1. On mount, call `JournalService.getEntry(id)`
2. Decrypt content using `encryption.ts` before displaying
3. If `audio_path` exists, show audio player — stream from `API_BASE_URL/uploads/filename`

4. Edit button navigates to Edit Entry Screen with entry data pre-loaded
5. Delete button shows confirmation alert: "Delete this entry? This cannot be undone."
6. On confirm delete, call `JournalService.deleteEntry(id)`
7. On success, navigate back to Journal List Screen

API Calls

GET /api/entries/:id

Response: { id, entry_date, mood, content (encrypted), audio_path, tags }

DELETE /api/entries/:id

Response: { message: "Entry deleted" }

Screen: Edit Entry Screen

Route: `/entry/:id/edit`

User Story

As a user, I want to edit an existing journal entry so I can correct mistakes or add more thoughts after saving.

UI Elements

- Same layout as New Entry Screen
- All fields pre-populated with existing entry data
- Save Changes button top right

Frontend Logic

1. Receive entry data from navigation params (already decrypted from Detail Screen)
2. Pre-populate mood selector, text input, and tag chips
3. User makes changes
4. On Save Changes:
 - Re-encrypt updated content with AES-256

- Call `JournalService.updateEntry(id, { mood, content: encrypted, tags })`
- On success, navigate back to Entry Detail Screen

API Call

PATCH `/api/entries/:id`

Body: `{ mood, content (encrypted), tags[] }`

Response: `{ id, entry_date, mood, content, tags, updated_at }`

API Routes: Journal Module

GET `/api/entries`

User Story: As a user browsing my journal, I want to fetch all my entries with optional filters so I can see the right entries on the list screen.

Method: GET

Route: `/api/entries`

Auth: Bearer token required

Query: `?date=2024-01-15 | ?mood=3 | ?tag=work | ?search=keyword`

Response: `[ { id, entry_date, mood, content, audio_path, tags, created_at } ]`

Step-by-step Logic:

1. Extract `userId` from JWT via `authMiddleware`
2. Build base query: `SELECT * FROM journal_entries WHERE user_id = $1`
3. If `date` query param present, append: `AND entry_date = $2`
4. If `mood` query param present, append: `AND mood = $2`
5. If `tag` query param present, append: `AND $2 = ANY(tags)`
6. If `search` query param present, append: `AND content ILIKE $2` with value `%keyword%`

7. Append `ORDER BY entry_date DESC`
8. Execute query with parameterized values
9. Return array (empty array if no results, not 404)

Error Responses:

- 401: Missing or invalid token
 - 500: Server error
-

POST /api/entries

User Story: As a user who just finished writing, I want to save my journal entry so it is stored securely and my streak is updated.

Method: POST

Route: /api/entries

Auth: Bearer token required

Body: { entry_date, mood, content, tags[], audio_path }

Response: { id, entry_date, mood, content, tags, created_at }

Step-by-step Logic:

1. Extract `userId` from JWT
2. Validate `entry_date`, `mood` (1-5), and `content` are present — return 400 if missing
3. Insert: `INSERT INTO journal_entries (user_id, entry_date, mood, content, audio_path, tags) VALUES ($1,$2,$3,$4,$5,$6) RETURNING *`
4. Call `streakService.updateStreak(userId, entry_date)` after successful insert
5. Return the newly created entry row

Error Responses:

- 400: Missing required fields
 - 401: Unauthorized
 - 500: Server error
-

GET /api/entries/:id

User Story: As a user tapping on an entry card, I want to fetch the full entry data so I can read it on the detail screen.

Method: GET

Route: /api/entries/:id

Auth: Bearer token required

Response: { id, entry_date, mood, content, audio_path, tags, created_at, updated_at }

Step-by-step Logic:

1. Extract `userId` from JWT
2. Query: `SELECT * FROM journal_entries WHERE id = $1 AND user_id = $2`
3. Including `user_id` in WHERE clause prevents users from accessing other users' entries
4. Return 404 if no row found
5. Return entry object

Error Responses:

- 401: Unauthorized
 - 404: Entry not found or does not belong to user
 - 500: Server error
-

PATCH /api/entries/:id

User Story: As a user editing an entry, I want to update the content, mood, or tags so my journal reflects my latest thoughts.

Method: PATCH

Route: /api/entries/:id

Auth: Bearer token required

Body: { mood, content, tags[] }

Response: { id, entry_date, mood, content, tags, updated_at }

Step-by-step Logic:

1. Extract `userId` from JWT
2. Validate at least one of `mood`, `content`, or `tags` is present in body — return 400 if all missing
3. Build dynamic SET clause from provided fields only
4. Always include `updated_at = NOW()` in SET clause
5. Query: `UPDATE journal_entries SET ... WHERE id = $1 AND user_id = $2 RETURNING *`
6. Return 404 if no row updated (entry not found or wrong user)
7. Return updated entry row

Error Responses:

- 400: No fields to update
 - 401: Unauthorized
 - 404: Entry not found
 - 500: Server error
-

DELETE /api/entries/:id

User Story: As a user who wants to remove an entry, I want to permanently delete it so it no longer appears in my journal.

Method: DELETE

Route: /api/entries/:id

Auth: Bearer token required

Response: { message: "Entry deleted" }

Step-by-step Logic:

1. Extract `userId` from JWT
2. Query: `DELETE FROM journal_entries WHERE id = $1 AND user_id = $2 RETURNING id`
3. Including `user_id` ensures users can only delete their own entries
4. Return 404 if no row deleted
5. If entry had `audio_path`, delete the file from disk: `fs.unlinkSync(audio_path)`
6. Return success message

Error Responses:

- 401: Unauthorized
 - 404: Entry not found
 - 500: Server error
-

MODULE 3: VOICE AND TRANSCRIPTION MODULE

Overview

Handles audio recording on device and transcription using self-hosted Whisper.cpp. User speaks, audio is uploaded to the server, Whisper converts it to text, and the text is inserted into the entry input field. No OpenAI API key needed.

Backend Files

- `src/routes/transcribe.js`
- `src/controllers/transcribeController.js`
- `src/middleware/uploadMiddleware.js`
- `src/services/whisperService.js`

Frontend Files

- `screens/NewEntryScreen.tsx` (mic button)
 - `components/VoiceOverlay.tsx`
 - `components/WaveformAnimation.tsx`
 - `services/AudioService.ts`
-

Screen: Voice Recording Overlay

Route: Modal overlay on New Entry Screen

User Story

As a user who prefers speaking over typing, I want to tap a mic button, speak my thoughts, and have them automatically transcribed into my journal entry.

UI Elements

- Full screen semi-transparent overlay (shown on mic tap)
- Centered microphone icon with pulse animation
- Animated waveform below icon (reacts to audio input)
- Live duration timer (e.g. 00:23)
- Large Stop button
- Cancel button (top right)
- Loading spinner with "Transcribing..." text (shown after Stop)

Frontend Logic

1. User taps mic button on New Entry Screen
2. Overlay appears and `AudioService.startRecording()` called immediately
3. Timer starts counting up
4. Waveform animates to indicate active recording
5. User taps Stop button:
 - `AudioService.stopRecording()` returns local audio file URI
 - Show "Transcribing..." spinner
 - Call `AudioService.uploadAndTranscribe(fileURI)` which POSTs to `/api/transcribe`
6. On response, insert transcript text into the text input on New Entry Screen
7. Dismiss overlay
8. On Cancel, stop recording and delete temp file, dismiss overlay without inserting text

API Call

POST `/api/transcribe`

Body: multipart/form-data, field name "audio", file type .wav or .m4a

Response: { transcript: "this is what the user said" }

API Routes: Transcription Module

POST `/api/transcribe`

User Story: As a user who just recorded their voice, I want the server to transcribe my audio into text so I can see my spoken words in my journal entry without typing.

Method: POST

Route: /api/transcribe

Auth: Bearer token required

Body: multipart/form-data — field: "audio" (audio file)

Response: { transcript: "transcribed text string" }

Step-by-step Logic:

1. `uploadMiddleware` (multer) receives the audio file and saves it to `./uploads/` with a unique filename
2. Extract saved file path from `req.file.path`
3. Convert audio to WAV format if needed (Whisper requires WAV): use `ffmpeg` via `child_process`
4. Call `whisperService.transcribe(filePath)`:
 - Runs: `./whisper.cpp/main -m ./models/ggml-base.en.bin -f audio.wav -otxt`
 - Whisper writes output to `audio.wav.txt` in same directory
 - Read the `.txt` file content
5. Delete both the audio file and the `.txt` file from disk after reading
6. Return `{ transcript: text.trim() }`

whisperService.js implementation:

```
const { exec } = require('child_process');
```

```
const fs = require('fs');
```

```
async function transcribe(audioPath) {
```

```
  return new Promise((resolve, reject) => {
```

```
    const cmd = `${process.env.WHISPER_BINARY} -m  
${process.env.WHISPER_MODEL_PATH} -f ${audioPath} -otxt`;
```

```
    exec(cmd, (error) => {
```

```
    if (error) return reject(error);

    const txtPath = audioPath + '.txt';

    const text = fs.readFileSync(txtPath, 'utf8');

    fs.unlinkSync(audioPath);

    fs.unlinkSync(txtPath);

    resolve(text.trim());

  });

});

}

module.exports = { transcribe };
```

Error Responses:

- 400: No audio file in request
 - 401: Unauthorized
 - 500: Whisper failed or file error
-

MODULE 4: MOOD MODULE

Overview

Mood is an integer from 1 to 5 stored with every journal entry. This module covers the mood picker UI component and the mood filter functionality on the Journal List Screen.

Frontend Files

- `components/MoodSelector.tsx` — 5-emoji row picker used in New Entry and Edit Entry screens

- `components/MoodChip.tsx` — small emoji filter chip used in Journal List Screen

Mood Scale

Value	Emoji	Label
-------	-------	-------

1		Awful
---	---	-------

2		Bad
---	---	-----

3		Okay
---	---	------

4		Good
---	---	------

5		Amazing
---	---	---------

User Story

As a user saving an entry, I want to select my mood from a simple 5-emoji scale so I can track my emotional state without writing about it explicitly.

MoodSelector Component Logic

1. Renders 5 emoji buttons in a horizontal row
 2. Tapping an emoji sets the selected value and highlights it with a background circle
 3. Previously selected value loses highlight
 4. Selected mood value is passed up to parent screen via `onMoodSelect(value)` callback
 5. Mood is required before saving an entry — parent validates this
-

MODULE 5: AI SUMMARY MODULE

Overview

Generates weekly AI-powered summaries using self-hosted Ollama with Llama 3.1. Every Sunday (or on demand), the user can generate a summary of their week that includes emotional patterns, recurring themes, and notable moments. No API key needed.

Backend Files

- `src/routes/summaries.js`
- `src/controllers/summariesController.js`
- `src/services/ollamaService.js`

Frontend Files

- `screens/InsightsScreen.tsx`
 - `components/WeeklySummaryCard.tsx`
 - `components/MoodChart.tsx`
 - `components/MoodPieChart.tsx`
 - `services/SummaryService.ts`
-

Screen: Insights Screen

Route: `/insights`

User Story

As a user who wants to understand my emotional patterns, I want to see a visual mood chart for the week and an AI-generated summary of my entries so I can reflect on how my week went.

UI Elements

- Week selector arrows (navigate to previous weeks)
- Current week date range label (e.g. "Jan 13 - Jan 19")
- Mood trend line chart showing mood value per day for 7 days
- Mood distribution pie chart (% of each mood level this week)
- Entry count and average mood stats row

- AI Weekly Summary card:
 - If generated: shows summary paragraph and top theme tags
 - If not generated: shows "Generate This Week's Summary" button
- Loading skeleton while fetching

Frontend Logic

1. On mount, calculate current week start (Monday) date
2. Call `JournalService.getEntries()` filtered by week date range
3. Render mood data in charts using Victory Native library
4. Call `SummaryService.getSummary(week_start)` to check if summary exists
5. If summary exists, render `WeeklySummaryCard` with summary and themes
6. If no summary, show generate button
7. On Generate tap:
 - Show loading spinner on button
 - Call `SummaryService.generateSummary(week_start)`
 - On response, render summary card
8. Week selector arrows recalculate week_start and re-fetch all data

API Calls

GET `/api/summaries?week_start=2024-01-15`

Response: { id, week_start, summary, top_themes, avg_mood } or null

POST `/api/summaries/generate`

Body: { week_start: "2024-01-15" }

Response: { summary, top_themes, avg_mood }

API Routes: Summary Module

GET `/api/summaries`

User Story: As a user opening the Insights screen, I want the app to check if a summary already exists for this week so I do not have to generate it again.

Method: GET

Route: /api/summaries

Auth: Bearer token required

Query: ?week_start=2024-01-15

Response: { id, week_start, summary, top_themes, avg_mood } or { data: null }

Step-by-step Logic:

1. Extract `userId` from JWT
2. Validate `week_start` query param present — return 400 if missing
3. Query: `SELECT * FROM weekly_summaries WHERE user_id = $1 AND week_start = $2`
4. Return the summary row if found
5. Return { `data: null` } if not found (not a 404 — absence of summary is a valid state)

Error Responses:

- 400: Missing week_start param
- 401: Unauthorized
- 500: Server error

POST /api/summaries/generate

User Story: As a user who wants insight into my week, I want the server to read my entries, send them to the AI, and return a human-readable summary so I can reflect without re-reading every entry.

Method: POST

Route: /api/summaries/generate

Auth: Bearer token required

Body: { week_start: "2024-01-15" }

Response: { summary, top_themes, avg_mood }

Step-by-step Logic:

1. Extract `userId` from JWT
2. Validate `week_start` is present and is a valid date — return 400 if not
3. Calculate `week_end` = `week_start` + 6 days
4. Query all entries: `SELECT entry_date, mood, content FROM journal_entries WHERE user_id = $1 AND entry_date BETWEEN $2 AND $3`
5. If 0 entries found, return 400: "No entries found for this week"
6. Calculate `avg_mood` from entries array
7. Decrypt each entry content before sending to Ollama (content is stored encrypted)
8. Build Ollama prompt:

You are a compassionate personal journal analyst.

Below are journal entries from the week of [week_start] to [week_end].

[For each entry: Date: YYYY-MM-DD | Mood: X/5 | Entry: decrypted content]

Write a 3-4 sentence summary of this person's emotional week.

Focus on emotional patterns, recurring themes, and key moments.

Be warm, insightful, and specific to what they wrote.

Then list 3-5 single-word themes from this week.

Respond ONLY as valid JSON with no extra text:

```
{  
  "summary": "...",  
  "themes": ["theme1", "theme2", "theme3"]  
}
```

9. Call `ollamaService.generate(prompt)`
10. Parse JSON from Ollama response
11. Upsert into `weekly_summaries`: `INSERT INTO weekly_summaries ... ON CONFLICT (user_id, week_start) DO UPDATE SET summary = $3, top_themes = $4, avg_mood = $5`
12. Return `{ summary, top_themes, avg_mood }`

ollamaService.js implementation:

```
async function generate(prompt) {  
  
  const response = await fetch(`${process.env.OLLAMA_URL}/api/generate`, {  
  
    method: 'POST',  
  
    headers: { 'Content-Type': 'application/json' },  
  
    body: JSON.stringify({  
  
      model: process.env.OLLAMA_MODEL,  
  
      prompt: prompt,  
  
      stream: false  
  
    })  
  
  });  
  
  const data = await response.json();  
  
  return data.response;  
  
}
```

```
module.exports = { generate };
```

Error Responses:

- 400: Missing `week_start` or no entries found
- 401: Unauthorized

- 500: Ollama error or JSON parse failure
-

MODULE 6: STREAK MODULE

Overview

Tracks the user's daily journaling streak. Every time a new entry is saved, the streak service checks if the user journaled yesterday and increments or resets accordingly.

Backend Files

- `src/routes/streaks.js`
- `src/controllers/streaksController.js`
- `src/services/streakService.js`

Frontend Files

- `components/StreakBadge.tsx`
 - `services/StreakService.ts`
-

StreakBadge Component

User Story

As a user on the Home Screen, I want to see my current journaling streak so I feel motivated to keep writing every day.

UI Logic

- Displays flame emoji and current streak number (e.g. "🔥 7")
 - If streak is 0, shows "Start your streak today"
 - Tapping badge shows a tooltip: "Longest streak: X days"
-

API Routes: Streak Module

GET /api/streaks

User Story: As a user opening the app, I want my streak data loaded so I can see how many consecutive days I have journaled.

Method: GET

Route: /api/streaks

Auth: Bearer token required

Response: { current_streak, longest_streak, last_entry_date }

Step-by-step Logic:

1. Extract `userId` from JWT
2. Query: `SELECT current_streak, longest_streak, last_entry_date FROM streaks WHERE user_id = $1`
3. Return 404 if no streak row found (should not happen — row created on register)
4. Return streak object

Error Responses:

- 401: Unauthorized
- 404: Streak record not found
- 500: Server error

streakService.js Logic

Called automatically after every successful POST /api/entries

```
async function updateStreak(userId, entryDate) {
```

```
  const result = await db.query(
```

```
    'SELECT current_streak, longest_streak, last_entry_date FROM streaks WHERE user_id = $1',
```

```
    [userId]
```

```
);

const streak = result.rows[0];

const today = new Date(entryDate);

const yesterday = new Date(today);

yesterday.setDate(yesterday.getDate() - 1);

const yesterdayStr = yesterday.toISOString().split('T')[0];

// Already journaled today, no change needed

if (streak.last_entry_date === entryDate) return;

let newStreak;

if (streak.last_entry_date === yesterdayStr) {

  // Consecutive day — increment

  newStreak = streak.current_streak + 1;

} else {

  // Gap in entries — reset to 1

  newStreak = 1;

}

const longestStreak = Math.max(newStreak, streak.longest_streak);

await db.query(

  `UPDATE streaks

  SET current_streak = $1, longest_streak = $2, last_entry_date = $3, updated_at = NOW()
```

```
WHERE user_id = $4`,  
[newStreak, longestStreak, entryDate, userId]  
);  
}  
  
module.exports = { updateStreak };
```

MODULE 7: NOTIFICATIONS MODULE

Overview

Handles daily reminder notifications that prompt the user to journal each day. Uses Expo's local notification system — no push notification server needed. Notifications are scheduled entirely on device.

Backend Files

- `src/routes/settings.js`
- `src/controllers/settingsController.js`

Frontend Files

- `services/NotificationService.ts`
- `screens/SettingsScreen.tsx`

Screen: Settings Screen

Route: `/settings`

User Story

As a user who wants to build a journaling habit, I want to set a daily reminder so the app notifies me at my chosen time to write my entry.

UI Elements

- Profile section: full name and email (read-only)
- Daily Reminder toggle switch
- Time picker (appears when toggle is on)
- Export Journal button (generates PDF of all entries)
- Delete Account button (red, with confirmation)
- Logout button

Frontend Logic

1. On mount, call `StreakService.getSettings()` to load current preferences
2. Render toggle state from `reminder_enabled`
3. On toggle ON:
 - Request notification permission via `NotificationService.requestPermissions()`
 - Show time picker defaulting to 9:00 AM
 - Schedule local notification via `NotificationService.scheduleDailyReminder(time)`
 - Call `PATCH /api/settings` with `{ reminder_enabled: true, reminder_time: time }`
4. On toggle OFF:
 - Cancel all scheduled notifications via `NotificationService.cancelAll()`
 - Call `PATCH /api/settings` with `{ reminder_enabled: false }`
5. Logout: clear JWT from SecureStore, clear authStore, navigate to Login Screen
6. Delete Account: show confirmation alert, call `DELETE /api/auth/account`, clear local data

API Calls

GET /api/settings

Response: { reminder_enabled, reminder_time }

PATCH /api/settings

Body: { reminder_enabled, reminder_time }

Response: { reminder_enabled, reminder_time, updated_at }

API Routes: Settings Module

GET /api/settings

User Story: As a user opening the Settings screen, I want to load my current preferences so the toggles reflect what I already set up.

Method: GET

Route: /api/settings

Auth: Bearer token required

Response: { reminder_enabled, reminder_time }

Step-by-step Logic:

1. Extract `userId` from JWT
2. Query: `SELECT reminder_enabled, reminder_time FROM user_settings WHERE user_id = $1`
3. Return 404 if row not found (should not happen — created on register)
4. Return settings object

PATCH /api/settings

User Story: As a user changing my reminder preferences, I want the server to save my new settings so they persist across devices and app reinstalls.

Method: PATCH

Route: /api/settings

Auth: Bearer token required

Body: { reminder_enabled, reminder_time }

Response: { reminder_enabled, reminder_time, updated_at }

Step-by-step Logic:

1. Extract `userId` from JWT
2. Validate at least one field present in body
3. Build dynamic SET clause from provided fields
4. Always include `updated_at = NOW()`
5. Query: `UPDATE user_settings SET ... WHERE user_id = $1 RETURNING *`
6. Return updated settings row

Error Responses:

- 400: No fields to update
 - 401: Unauthorized
 - 500: Server error
-

ONBOARDING SCREEN

Route: `/onboarding`

User Story

As a first-time user, I want to see a brief introduction to what the app does so I understand its value before creating an account.

UI Elements

- 3 swipeable slides with illustration, headline, and subtitle each:
 - Slide 1: Illustration of journal / "Your private journal, powered by AI"
 - Slide 2: Mic icon / "Speak your thoughts, we transcribe them"
 - Slide 3: Chart icon / "Get weekly insights about your mind"
- Dot pagination indicator
- Skip button (top right)
- Next button (bottom right)
- Get Started button (on last slide only)

Frontend Logic

1. Check AsyncStorage for `onboarding_complete` key on app launch
2. If key exists, skip Onboarding and navigate directly to Login

3. If not, show Onboarding
 4. On Get Started or Skip, set `onboarding_complete: true` in AsyncStorage
 5. Navigate to Register Screen
-

SERVER SETUP

Oracle Cloud Free Tier Setup Commands

Step 1: Update system packages

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Install Node.js 20

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo bash -
```

```
sudo apt install nodejs -y
```

```
node --version
```

Step 3: Install PostgreSQL

```
sudo apt install postgresql postgresql-contrib -y
```

```
sudo systemctl start postgresql
```

```
sudo systemctl enable postgresql
```

```
sudo -u postgres psql -c "CREATE USER reflekt WITH PASSWORD 'your_password';"
```

```
sudo -u postgres psql -c "CREATE DATABASE reflekt OWNER reflekt;"
```

Step 4: Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
```

```
ollama pull llama3.1
```

ollama serve &

Step 5: Install Whisper.cpp

sudo apt install build-essential ffmpeg -y

git clone <https://github.com/ggerganov/whisper.cpp>

cd whisper.cpp && make

bash ./models/download-ggml-model.sh base.en

cd ..

Step 6: Clone and run backend

git clone your-repo-url backend

cd backend

npm install

cp .env.example .env

Edit .env with your values

node src/migrations/run.js

npm start

Step 7: Install PM2 for process management

npm install -g pm2

pm2 start src/index.js --name reflekt-api

pm2 startup

pm2 save

3-WEEK DEVELOPMENT ROADMAP

Week 1: Foundation

Day	Backend Tasks	Frontend Tasks
Day 1	Oracle server setup, PostgreSQL install, run all 5 migrations	Expo project init, folder structure, install dependencies
Day 2	Auth routes: register, login, me. JWT middleware. bcrypt setup	Register and Login screens wired to API. JWT stored in SecureStore
Day 3	Entries routes: GET all, POST create. Streak service stub	Home Screen, bottom tab nav, streak badge display
Day 4	Entries routes: GET by id, PATCH, DELETE. Streak service complete	New Entry Screen: mood selector, text input, save to API
Day 5	Transcribe route: multer upload, Whisper.cpp integration	Voice overlay, expo-av recording, transcription wired into entry

Week 2: Core Features

Day	Backend Tasks	Frontend Tasks
Day 6	Settings routes: GET and PATCH. Auth/me complete	Journal List Screen: fetch, filter chips, search

Day 7	Audio file serving via static route	Entry Detail Screen: decrypt, audio playback, delete
Day 8	Test all entry routes with Postman	Edit Entry Screen: pre-fill, update flow
Day 9	AES encryption review, ensure content never stored plaintext	Encryption.ts wired into all save and fetch flows
Day 10	Streak logic edge cases: same day duplicate, long gap	Streak badge updates live after new entry saved

Week 3: AI and Polish

Day	Backend Tasks	Frontend Tasks
Day 11	Ollama service: prompt engineering, JSON parse	Insights Screen: week selector, mood data fetch
Day 12	Summary generate route: full flow tested	Mood line chart and pie chart with Victory Native
Day 13	Summary fetch route, upsert on regenerate	Weekly summary card, top themes, generate button
Day 14	Final API error handling review	Settings Screen: notifications, reminder scheduler

Day 15	Load test Ollama and Whisper on Oracle server	UI polish: spacing, typography, empty states
Day 16	PM2 process config, nginx reverse proxy setup	Error handling: network errors, loading states
Day 17	End-to-end API tests	End-to-end app flow testing on physical device
Day 18	Production .env hardening	APK build for Android, TestFlight for iOS

TOTAL COST

Item	Cost
Oracle Cloud server (24GB RAM)	Free forever
PostgreSQL (self-hosted)	Free
Ollama + Llama 3.1 (self-hosted)	Free
Whisper.cpp (self-hosted)	Free
React Native + Expo	Free

Node.js + Express	Free
-------------------	------

expo-av, Victory Native, Zustand	Free
----------------------------------	------

Total monthly cost	\$0
--------------------	-----

Reflekt AI Journal PRD v3.0 — Node.js + Express + PostgreSQL + Ollama + Whisper.cpp —
100% Free Stack